Use code with caution.

- View your module's `printk` startup message:

  bash

  ```
  dmesg | tail -n 10
  ```

  Use code with caution.

Did running the **`KBUILD_CFLAGS`** command clear up the load error? If you check **`lsmod | grep hello`**, do you see your module listed in the output?

Shared

0 files

Show all

rmmod <module name> causing freeze

AI Mode reply for rmmod <module name> causing freeze

When `rmmod` causes your entire system to freeze or hang, it means the kernel has encountered a **deadlock** or a **kernel panic** inside your module's cleanup function (`module_exit`).

Since you are writing an interrupt module (`first_interrupt`), a freeze during `rmmod` is almost always caused by forgetting to free a system resource before the module unloads, causing the kernel to wait forever or crash.

Here are the most common coding mistakes that cause an `rmmod` freeze and how to fix them:

1. Forgetting to Free the IRQ (Interrupt Line)

If your module requests a hardware interrupt in the initialization phase using `request_irq()`, you **must** release it in your cleanup function using `free_irq()`. If you don't, the kernel keeps trying to send hardware signals to a function pointer that no longer exists in memory, causing an instant freeze.

❌ The Buggy Code:

c

```
static void __exit my_exit(void) {
    printk(KERN_INFO "Goodbye!\n");
    // Freezing here because the IRQ is still registered!
}
```

Use code with caution.

** By-the-Book Fix:**

c

```
static void __exit my_exit(void) {
    // 1. You must explicitly match the IRQ number and the dev_id pointer used in request_irq
    free_irq(MY_IRQ_NUMBER, my_dev_id_pointer);
    printk(KERN_INFO "Interrupt freed. Safe to unload.\n");
}
```

Use code with caution.

2. Leaving a Concurrency Tool (like a Timer or Workqueue) Active

If your interrupt fires a **Kernel Timer**, a **Tasklet**, or a **Workqueue** to handle background tasks, and that task is still running or scheduled when you type `rmmod`, the kernel will deadlock.

You must explicitly cancel them in your exit function:

c

```
static void __exit my_exit(void) {
    del_timer_sync(&my_timer);       // Wait for timers to finish
    cancel_work_sync(&my_work);      // Wait for workqueues to finish
    tasklet_kill(&my_tasklet);       // Stop scheduled tasklets

    free_irq(MY_IRQ_NUMBER, my_dev_id_pointer);
}
```

Use code with caution.

3. Forgetting to Unregister Devices or Drivers

If you created a `/dev/` file or registered a character device (`register_chrdev` or `cdev_add`), you must undo it. Failing to unregister these leaves dangling pointers in the kernel VFS layer.

c

```
// Ensure these matches are in your exit function if you used them:
unregister_chrdev(major_number, "my_device");
device_destroy(my_class, dev_no);
class_destroy(my_class);
```

Use code with caution.

How to Safely Debug a Frozen VM

Because the virtual machine completely locks up, you cannot read standard terminal logs. Use these tactics to find the error:

1. **Keep `dmesg -w` open:** Before you run `sudo rmmod`, open a second terminal window in VirtualBox and run:

   bash

   ```
   sudo dmesg -w
   ```

   Use code with caution.

   Leave it visible. When you trigger `rmmod` in the first window, the second window will capture the final trace details right before the freeze.
2. **Check for an Infinite Loop:** Look closely at your cleanup function. Ensure you do not have a `while` loop waiting for a condition that can never be met while unloading.

Could you paste your module's **`static void __exit`** cleanup function? I can check it over to point out exactly which resource is missing its release call or causing the deadlock!

Copied to clipboardFailed to copy to clipboard. Try again later.

Shared

0 files

- Stack Overflow

  "Segmentation fault" when `rmmod` or `modprobe -r`

  Jun 11, 2015 — The kernel is compiled with module unloading (both regular and forced) support enabled. What can be the possible cause of this iss...

- Stack Overflow

  Linux block device module is hanging when unloading with rmmod (...

  Jun 15, 2019 — I figured out, that in the module unload function, the call to del_gendisk caused the hanging. This mechanism hinders you to unloa...

Show all

On exit. I am doing this. free_irq(IRQ_NUMBER, (void *)(dev_id))

AI Mode reply for On exit. I am doing this. free_irq(IRQ_NUMBER, (void *)(dev_id))

Your syntax for `free_irq` looks perfect, but a freeze here usually means a structural mismatch between how the interrupt was **requested** (`request_irq`) and how it is being **freed** (`free_irq`).

The Linux kernel matches interrupts using a combination of the `IRQ_NUMBER` and the `dev_id` pointer. If they don't line up exactly, `free_irq` can fail silently, or worse, execute a **Null Pointer Dereference** or cause a **deadlock** that completely freezes the system.

Check for these three hidden traps in your implementation:

1. The `dev_id` Pointer Mismatch (Most Common)

The value of the pointer passed into `free_irq` **must match the exact same memory address** used during `request_irq`.

- If you passed `&dev_id` (the address of the variable) in `request_irq`, you must pass `&dev_id` in `free_irq`.
- If you passed an integer casted to a pointer (like `(void *)1234`) in `request_irq`, the exact same numeric value must be passed during the exit phase.
- **The Rule:** If the kernel cannot find a registered interrupt with that exact `IRQ_NUMBER` **and** that exact `dev_id` address, it won't free it. When the module unloads, the interrupt handler becomes a dangling pointer. The next time the hardware ticks, the kernel jumps into empty memory and crashes.

2. Is the Interrupt Handler Still Running?

If you are testing your module by triggering a fast, high-frequency interrupt (like a mouse move, keyboard stroke, or network packet), the interrupt might be firing *at the exact microsecond* you run `rmmod`.

`free_irq` is a synchronous call—it will spin-lock and **wait forever** if the interrupt service routine (ISR) is currently running and never relinquishes control. Ensure your ISR returns `IRQ_HANDLED` immediately and doesn't get stuck in a long loop.

3. Check for a Shared Interrupt Conflict (`IRQF_SHARED`)

If you registered the interrupt line using the `IRQF_SHARED` flag (meaning other system devices like your keyboard or network card share this line):

- You **cannot** pass `NULL` as a `dev_id`. Shared interrupts strictly require a unique, valid memory address (usually a pointer to your device's private structure variable).
- If you pass a generic pointer that another driver is already using, `free_irq` might accidentally try to unregister the wrong driver, causing the kernel to panic.

Step-by-Step Validation Code

To see if your exit function is even reaching the `free_irq` line, add temporary tracking prints around it. Update your cleanup function to look like this:

c

```
static void __exit my_module_exit(void) {
    printk(KERN_INFO "Module: Attempting to free IRQ %d with dev_id %p\n", IRQ_NUMBER, (void *)dev_id);

    // Perform the free
    free_irq(IRQ_NUMBER, (void *)dev_id);

    printk(KERN_INFO "Module: IRQ successfully freed! Safe to exit.\n");
}
```

Use code with caution.

Recompile the module, run `sudo dmesg -w` in a separate side-by-side terminal, and trigger the `rmmod`.

- **If it stops *before* printing "IRQ successfully freed":** The kernel is deadlocking inside `free_irq` because of a pointer mismatch or a running handler lock.
- **If it prints "Safe to exit" but *then* freezes:** The interrupt wasn't actually unregistered correctly, and the kernel is crashing the moment the module's memory space is erased.

Could you show the exact line of code where you call **`request_irq`** in your initialization function, along with how **`dev_id`** is declared? Comparing the request and the free side-by-side will pinpoint the exact mismatch.

Copied to clipboardFailed to copy to clipboard. Try again later.

Shared

0 files

Show all

request_irq(IRQ_NUMBER, handler, IRQF_SHARED, "my_interrupt", &(dev_id));

AI Mode reply for request_irq(IRQ_NUMBER, handler, IRQF_SHARED, "my_interrupt", &(dev_id));

There it is! You have found the exact cause of the system freeze.

You have a **pointer level mismatch** between how you are requesting the interrupt and how you are freeing it.

The Breakdown of the Bug

- In `request_irq`, you passed **`&(dev_id)`** (the *address* of your pointer variable, which means a pointer-to-a-pointer or `void **`).
- In `free_irq`, you passed **`(void *)(dev_id)`** (the *value* stored inside the pointer variable, which is a `void *`).

Because the memory addresses do not match, the Linux kernel cannot find your registration in its internal interrupt table. It leaves your handler active in memory. The moment `rmmod` finishes, your module's code is wiped from RAM. A split second later, when the shared interrupt line triggers, the kernel jumps to the old address, finds nothing but empty space, and **instantly deadlocks or panics the CPU**.

The Fix

Update your `free_irq` call inside your exit function to pass the exact same address reference (**`&dev_id`**) you used during the setup phase:

c

```
static void __exit my_module_exit(void) {
    // Pass the address of dev_id, exactly like request_irq did
    free_irq(IRQ_NUMBER, &dev_id);

    printk(KERN_INFO "Interrupt successfully unregistered!\n");
}
```

Use code with caution.

A Cleaner Best-Practice Approach

For shared interrupts (`IRQF_SHARED`), it is standard practice to pass a pointer to a global or static structure, or a dummy token variable, rather than tracking references of addresses.

For instance, you can simply declare an integer identifier token:

c

```
static int my_dev_id = 42; // A unique token for your module
```

Use code with caution.

Then use it cleanly without confusing address symbols (`&`):

c

```
// In your init function:
request_irq(IRQ_NUMBER, handler, IRQF_SHARED, "my_interrupt", &my_dev_id);
```

```
// In your exit function:
free_irq(IRQ_NUMBER, &my_dev_id);
```

Use code with caution.

Once you change `(void *)(dev_id)` to **`&dev_id`** (or your token pointer), does **`rmmod`** unload smoothly without freezing your VirtualBox machine?

Copied to clipboardFailed to copy to clipboard. Try again later.

Shared

0 files

Show all

actually i declared void * dev_id;

AI Mode reply for actually i declared void * dev_id;

Ah, thank you for clarifying! Since `dev_id` is declared as a **`void *`**, it is already a pointer variable.

By passing **`&dev_id`** into `request_irq`, you were passing the memory address of the pointer variable itself (a `void **`). However, if that variable was allocated on the stack (inside your initialization function) rather than globally, its memory address vanished as soon as the init function ended.

Because `free_irq` could not find a matching `void **` registration, the kernel panicked during the unload.

The Final, Correct Fix

To make this safe and reliable, keep `dev_id` as a global or static variable, and pass its **value** (not its address) to **both** functions.

Here is exactly how your code should look:

1. Declare it globally at the top of your file:

c

```
static void *dev_id = (void *)1234; // Give it a dummy numerical value token
```

Use code with caution.

2. In your Initialization Function:

Pass `dev_id` directly without the `&` symbol.

c

```
static int __init my_module_init(void) {
    // Pass the value of dev_id directly
    int result = request_irq(IRQ_NUMBER, handler, IRQF_SHARED, "my_interrupt", dev_id);
    if (result) {
        printk(KERN_ERR "Failed to reserve IRQ\n");
        return result;
    }
    return 0;
}
```

Use code with caution.

3. In your Exit Function:

Pass `dev_id` directly without any extra pointer casts.

c

```
static void __exit my_module_exit(void) {
```

```
    // Pass the exact same value to unregister it
    free_irq(IRQ_NUMBER, dev_id);
    printk(KERN_INFO "Interrupt successfully unregistered!\n");
}
```

Use code with caution.

Why this fixes the freeze

By using `dev_id` directly in both places, the kernel registers the interrupt under the identifier `1234`. When `rmmod` is called, `free_irq` looks for `1234`, finds a perfect match, and cleanly safely detaches your handler from the kernel before wiping the memory.

Once you remove the `&` from your **`request_irq`** call and keep the arguments perfectly matched, does **`rmmod`** exit cleanly without freezing your system?

Copied to clipboardFailed to copy to clipboard. Try again later.

Shared

0 files

Show all

Shared

0 files